Reviewer Pack — Noncommutative L^p Geometric Entropy of Secant Varieties and...

Entry 3c109daf652d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 17:55:42 UTC

Noncommutative L^p Geometric Entropy of Secant Varieties and Communication Complexity of Disjointness

Entry ID: `3c109daf652d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 17:55:42 UTC

1. Conjecture

Field A (mathematical branch): noncommutative geometry of L^p spaces **Field B** (complexity object): Communication complexity of DISJOINTNESS

Statement:

> {'st1': 'The noncommutative geometric entropy of the secant variety associated with a given set X is lower-bounded by Ω(n) for the communication complexity of the partial function defined by X.', 'st2': 'Formally, for a set X, let V_X be its corresponding secant variety and H_μ(V_X) be its noncommutative L^p geometric entropy. Then H_μ(V_X) = Ω(n).', 'st3': 'This lower bound holds for all n ≤ 40 and can be computed in less than 240 seconds using a Python implementation of the associated geometric and algebraic operations.'}

Rationale (proposer's reasoning):

> {'r1': 'The noncommutative L^p geometry provides a framework to study geometric structures that are not well captured by classical geometry, which might reveal hidden complexities in the communication complexity of disjointness.', 'r2': 'Secant varieties are known to have intricate geometric properties that could potentially be exploited to derive strong lower bounds for communication complexity problems.', 'r3': 'The proposed conjecture aims to bridge these two fields and explore new avenues for understanding the inherent difficulty of disjointness.'}

Taxonomy category: `COMM_DISJ` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `9c71ed6f2ce04685`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The noncommutative L^p geometric entropy of a secant variety V_X associated with set X is considered supported if H_μ(V_X) >= Ω(n), where n is the number of elements in X, and falsified if any seed produces H_μ(V_X) < Ω(n).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `intitle:noncommutative geometry AND L^p AND secant varieties AND communication complexity AND disjointness` - `title:secant variety AND noncommutative L^p geometric entropy AND communication complexity of disjointness` - `title:L^p geometric entropy AND secant variety AND communication complexity`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 40)
    X = set(range(n))

    # Constructive mapping for secant variety and noncommutative L^p geometric entropy
    def secant_variety(X):
        V_X = []
        for x in X:
            for y in X:
                if x != y:
                    V_X.append((x, y))
        return V_X

    def noncommutative_Lp_entropy(V_X):
        # Placeholder implementation; actual computation depends on the conjecture
        return len(V_X) / n

    V_X = secant_variety(X)
    H_mu_V_X = noncommutative_Lp_entropy(V_X)

    metric_name = "noncommutative_Lp_entropy"
    metric_value = H_mu_V_X
    instances_tested = 1
    conjecture_holds = H_mu_V_X >= n
```

```
    counterexample = "" if conjecture_holds else f"mapping_undefined for n={n}"

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
cture_holds': False, 'counterexample': 'mapping_undefined for n=7'}
TRIAL: {'metric_name': 'noncommutative_Lp_entropy', 'metric_value': 24.0, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'noncommutative_Lp_entropy', 'metric_value': 22.0, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'noncommutative_Lp_entropy', 'metric_value': 8.0, 'instances_tested': 1, 'con
TRIAL: {'metric_name': 'noncommutative_Lp_entropy', 'metric_value': 37.0, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'noncommutative_Lp_entropy', 'metric_value': 15.0, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'noncommutative_Lp_entropy', 'metric_value': 35.0, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'noncommutative_Lp_entropy', 'metric_value': 18.0, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'noncommutative_Lp_entropy', 'metric_value': 16.0, 'instances_tested': 1, 'co

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4f3fefe3.py", line 70, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4f3fefe3.py", line 70, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                              ~^^^^^^^^
KeyError: 'seed'
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which prevents a definitive evaluation of the conjecture. | next: Investigate and fix the crash in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15467
2	preregistration	ollama_remote	glm4:latest	0	0	9567
3	novelty	ollama_remote	glm4:latest	0	0	8559
4	novelty	ollama_remote	glm4:latest	0	0	10598
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14652
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7833
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7937
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7262
9	judge	ollama_remote	glm4:latest	0	0	12284

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 94158 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/3c109daf652d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3c109daf652d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/3c109daf652d.tar.gz` (if generated)